

LAB REPORT 1: SEARCHING & SORTING ALGORITHMS

Course: Design & Analysis of Algorithms

Student Name: Aryan Nautiyal
Roll Number: 251210015

Date: 2026-07-28
Compiler / OS: clang++ -03 (Linux / POSIX)

1. Experiment 1A: Linear Search vs. Binary Search

1.1 Aim & Objective

To implement Linear Search and Binary Search algorithms in C++, measure their execution time using `<chrono>` for varying input sizes (N), and evaluate their scalability.

1.2 Theoretical Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Linear Search	$O(1)$	$O(N)$	$O(N)$	$O(1)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$

1.3 C++ Source Code

```
#include "../util/test.hpp" //this helper file is posted at the end
#include <chrono>
#include <iostream>
#include <vector>

int binarySearch(std::vector<int> &arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] > target) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return -1;
}

int linearSearch(std::vector<int> &arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

int main() {
    std::cout << "\n===== Search Benchmarks =====\n";
    std::vector<size_t> testSizes = {1000, 5000, 10000, 100000};

    for (size_t N : testSizes) {
        std::vector<int> data = generateTestCase(N, InputType::SORTED);

        std::cout << "\n[N = " << N << "] First 10 elements (sorted): ";
        size_t printCount = std::min(<size_t>(10), data.size());
```

```

for (size_t i = 0; i < printCount; ++i) {
    std::cout << data[i] << (i == printCount - 1 ? "" : ", ");
}
std::cout << "\n";

// Select a target that exists in the array (near the end to test linear
// search)
int target = data[N - 5];

auto startLin = std::chrono::steady_clock::now();
int linIdx = linearSearch(data, target);
auto endLin = std::chrono::steady_clock::now();
std::chrono::duration<double, std::milli> linTime = endLin - startLin;

auto startBin = std::chrono::steady_clock::now();
int binIdx = binarySearch(data, target);
auto endBin = std::chrono::steady_clock::now();
std::chrono::duration<double, std::milli> binTime = endBin - startBin;

// Output Benchmark Results
std::cout << "Searching for Target Value: " << target << "\n";
std::cout << "  Linear Search → Index: " << linIdx
    << " | Time: " << linTime.count() << " ms\n";
std::cout << "  Binary Search → Index: " << binIdx
    << " | Time: " << binTime.count() << " ms\n";
}
return 0;
}

```

1.4 Terminal Output

```

$ ./binarylinear

===== Search Benchmarks =====

[N = 1000] First 10 elements (sorted): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Searching for Target Value: 996
  Linear Search → Index: 995 | Time: 0.006542 ms
  Binary Search → Index: 995 | Time: 0.000633 ms

[N = 5000] First 10 elements (sorted): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Searching for Target Value: 4996
  Linear Search → Index: 4995 | Time: 0.021 ms
  Binary Search → Index: 4995 | Time: 0.000625 ms

[N = 10000] First 10 elements (sorted): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Searching for Target Value: 9996
  Linear Search → Index: 9995 | Time: 0.041599 ms
  Binary Search → Index: 9995 | Time: 0.000554 ms

[N = 100000] First 10 elements (sorted): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Searching for Target Value: 99996
  Linear Search → Index: 99995 | Time: 0.459308 ms
  Binary Search → Index: 99995 | Time: 0.001083 ms

```

2. Experiment 1B: Insertion Sort

2.1 Aim & Objective

To implement the Insertion Sort algorithm and verify its correctness on small input arrays.

2.2 Theoretical Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$

2.3 C++ Source Code

```
#include "../util/test.hpp"
#include <iostream>
#include <vector>

void insertionSort(std::vector<int> &arr) {
    for (int i = 1; i < arr.size(); i++) {
        int j = i, key = arr[i];
        while (j > 0 && key < arr[j - 1]) {
            arr[j] = arr[j - 1];
            j--;
        }
        arr[j] = key;
    }
}

int main() {
    std::vector<int> test = {12, 35, 3, 1, 2};
    std::cout << "Logic test:\nbefore: ";
    for (int num : test) {
        std::cout << num << ", ";
    }
    insertionSort(test);
    std::cout << "\nafter: ";
    for (int num : test) {
        std::cout << num << ", ";
    }

    std::cout << "\nBenchmarks:\n";
    std::vector<size_t> testSizes = {1000, 5000, 10000};

    for (size_t N : testSizes) {
        std::vector<int> data = generateTestCase(N, InputType::RANDOM);
        std::cout << "First 10 numbers(array):";
        for (auto i = 0; i < 11; i++) {
            std::cout << data[i] << ", ";
        }
        std::cout << '\n';
        auto start = std::chrono::steady_clock::now();
        insertionSort(data);
        auto end = std::chrono::steady_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << "Size: " << N << " | Time: " << duration.count() << " ms\n ";
    }
    return 0;
}
```

2.4 Terminal Output

```
$ ./insertion_demo
Logic test:
before: 12, 35, 3, 1, 2,
after: 1, 2, 3, 12, 35,
Benchmarks:
First 10 numbers(array):348352, 705602, 650026, 451297, 850448, 634833, 302036, 870354,
```

```

995527, 698690, 466961,
Size: 1000 | Time: 0.287559 ms
First 10 numbers(array):882799, 164198, 615879, 596708, 686678, 348941, 191989, 829075,
161463, 172042, 314507,
Size: 5000 | Time: 6.61195 ms
First 10 numbers(array):499539, 537606, 410066, 481565, 486714, 448728, 905243, 645083,
719242, 817801, 785261,
Size: 10000 | Time: 26.8338 ms

```

3. Experiment 1C: QuickSort & Empirical Performance Analysis

3.1 Aim & Objective

To implement QuickSort, generate test datasets (random, sorted, reverse sorted) for varying input sizes N , and measure sorting runtime using `<chrono>`.

3.2 Theoretical Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space Complexity(Best and Average Case)	Space Complexity(Worst case)
QuickSort (Lomuto)	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$	$O(n)$

3.3 C++ Source Code

```

#include "../util/test.hpp"
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

// --- Lomuto Partition Scheme
int partition(std::vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int j = low;
    for (int i = low; i < high; i++) {
        if (arr[i] ≤ pivot) {
            std::swap(arr[i], arr[j]);
            j++;
        }
    }
    std::swap(arr[j], arr[high]);
    return j;
}

void quickSort(std::vector<int> &arr, int low, int high) {
    if (low < high) {
        int k = partition(arr, low, high);
        quickSort(arr, low, k - 1);
        quickSort(arr, k + 1, high);
    }
}

void printFirst10(const std::vector<int> &arr) {
    size_t count = std::min(size_t(10), arr.size());
    for (size_t i = 0; i < count; i++) {
        std::cout << arr[i] << (i == count - 1 ? "" : ", ");
    }
    std::cout << "\n";
}

int main() {
    std::cout << " Correctness Test \n";

```

```

std::vector<int> test = {3, 2, 21, 4, 1, 15, 8};

std::cout << "Before sorting: ";
printFirst10(test);

quickSort(test, 0, test.size() - 1);

std::cout << "After sorting: ";
printFirst10(test);
std::cout << "Is Sorted: "
          << (std::is_sorted(test.begin(), test.end()) ? "Yes" : "No")
          << "\n\n";

std::cout << " Benchmarks \n";
std::vector<size_t> testSizes = {1000, 5000, 10000};
std::vector<std::pair<std::string, InputType>> testTypes = {
    {"Random Data", InputType::RANDOM},
    {"Sorted Data", InputType::SORTED},
    {"Reverse Sorted", InputType::REVERSE_SORTED}};

for (auto &[name, type] : testTypes) {
    std::cout << "\nTesting " << name << ":\n";
    for (size_t N : testSizes) {
        if (type != InputType::RANDOM && N ≥ 10000) {
            std::cout << "Size: " << N << " | Skipped (Stack overflow risk)\n";
            continue;
        }
        std::vector<int> data = generateTestCase(N, type);
        auto start = std::chrono::steady_clock::now();
        quickSort(data, 0, data.size() - 1);
        auto end = std::chrono::steady_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << "Size: " << N << " | Time: " << duration.count() << " ms"
                  << " | Validated: "
                  << (std::is_sorted(data.begin(), data.end()) ? "Yes" : "No")
                  << "\n";
    }
}
return 0;
}

```

3.4 Terminal Output

```

$ ./quicksort_bench
Correctness Test
Before sorting: 3, 2, 21, 4, 1, 15, 8
After sorting:  1, 2, 3, 4, 8, 15, 21
Is Sorted: Yes

Benchmarks

Testing Random Data:
Size: 1000 | Time: 0.327196 ms | Validated: Yes
Size: 5000 | Time: 2.1363 ms | Validated: Yes
Size: 10000 | Time: 3.58181 ms | Validated: Yes

Testing Sorted Data:
Size: 1000 | Time: 2.53235 ms | Validated: Yes
Size: 5000 | Time: 28.7542 ms | Validated: Yes
Size: 10000 | Skipped (Stack overflow risk)

```

```
Testing Reverse Sorted:
Size: 1000 | Time: 0.483486 ms | Validated: Yes
Size: 5000 | Time: 10.4051 ms | Validated: Yes
Size: 10000 | Skipped (Stack overflow risk)
```

Utility files: for generation of array

1. test.hpp

```
#pragma once

#include <chrono>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED, DUPLICATES };
std::vector<int> generateTestCase(size_t size, InputType type);
```

2. test.cpp

```
#include "test.hpp"
#include <algorithm>
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
    std::vector<int> arr(size);

    switch (type) {
        case InputType::RANDOM: {
            std::random_device rd;
            std::mt19937 gen(rd());
            std::uniform_int_distribution<int> dist(1, 1000000);
            for (size_t i = 0; i < size; ++i)
                arr[i] = dist(gen);
            break;
        }
        case InputType::SORTED: {
            std::iota(arr.begin(), arr.end(), 1);
            break;
        }
        case InputType::REVERSE_SORTED: {
            std::iota(arr.rbegin(), arr.rend(), 1);
            break;
        }
        case InputType::DUPLICATES: {
            std::fill(arr.begin(), arr.end(), 42);
            break;
        }
    }
    return arr;
}
```